So sánh Delta Lake, Apache Iceberg và Apache Hudi dưới workload cập nhật nhiều trên cụm nhỏ cho dữ liệu IoT và khí tượng – thủy văn

Hà Văn Vinh, Đỗ Thị Lan · *Học viên cao học – Môn Big Data Analytics and Management (TLU 2026), Project P1*

Tên đề tài: Đánh giá thực nghiệm ba table format Lakehouse (Delta Lake, Apache Iceberg, Apache Hudi) dưới workload cảm biến có dữ liệu đến trễ, trên phần cứng nhỏ. **English:** Update-Heavy Workloads on Small Clusters: An Experimental Comparison of Delta Lake, Apache Iceberg and Apache Hudi for IoT and Hydro-Meteorological Data

Abstract. Các table format Lakehouse — Delta Lake, Apache Iceberg và Apache Hudi — bổ sung giao dịch ACID, time travel và schema evolution lên trên file Parquet, nhưng các đánh giá đã công bố (LHBench [3], LST-Bench [4]) chủ yếu chạy TPC-DS thiên về đọc trên cụm đám mây lớn. Dữ liệu giám sát mưa, mực nước, chất lượng nước lại khác: dữ liệu đến liên tục theo micro-batch, số đo đến trễ phải ghi đè hồi tố, và đôi khi cả một trạm-ngày bị xóa. Đề tài xây dựng **sensor-LST**, bộ sinh workload mã nguồn mở dựng trên bộ dữ liệu thủy văn TSM-Bench d1 [7], phát lại append, upsert dữ liệu trễ và xóa theo trạm, kèm ground truth để kiểm tra từng checkpoint. Ba format được đánh giá ở hai chế độ ghi Copy-on-Write (CoW) và Merge-on-Read (MoR; với Delta là deletion vectors), bốn tỷ lệ cập nhật trễ $u \in \{0, 1, 5, 20\}$% và ba chu kỳ compaction trên một máy 4 vCPU — tổng cộng 228 lần chạy, tất cả khớp ground truth. Mã nguồn, notebook và dữ liệu đo: https://github.com/vinhha195-oss/lakehouse-p1.

- **Có dữ liệu trễ hay không mới là yếu tố quyết định chi phí, không phải tỷ lệ.** Tổng chi phí tăng 1,3–3,8 lần khi chuyển từ chỉ append sang có dữ liệu trễ, rồi gần như không đổi từ u = 1% đến 20% (không compaction, chênh tối đa 8%).
- **Iceberg MoR rẻ nhất ở mọi $u > 0$:** rẻ hơn 2,6–4,3 lần so với mọi cấu hình Delta và Hudi; write amplification 1,0–1,2 so với 9,4–11 của CoW.
- **Compaction mỗi 20 batch là đủ:** đưa độ trễ đọc của Iceberg và Hudi MoR về gần CoW với 8–17 s; compaction mỗi 5 batch chỉ tốn thêm.
- **Deletion vectors của Delta tiết kiệm byte nhưng không tiết kiệm thời gian:** ghi ít hơn 8 lần về byte nhưng chậm hơn Delta CoW cả khi ghi lẫn khi đọc ở quy mô này.
- **Kết quả gắn với quy mô và phần cứng:** một node, đĩa cục bộ, dữ liệu khoảng 222 MB; không khái quát cho cụm đám mây lớn (mục Caveats).

1. Introduction

- *Bối cảnh:* kiến trúc Lakehouse [1] lưu bảng dưới dạng file cột mở trên kho lưu trữ rẻ và khôi phục các tính năng của kho dữ liệu (giao dịch, phiên bản, data skipping) bằng một lớp metadata. Ba format chiếm ưu thế là Delta Lake [2], Apache Iceberg [15] và Apache Hudi [16]; trọng tâm nghiên cứu đã chuyển từ "có nên dùng" sang "đánh đổi thiết kế": cách ghi thay đổi cấp dòng, cách tổ chức metadata, cách compaction file nhỏ [3, 4, 5].
- *Vấn đề:* dữ liệu giám sát khí tượng – thủy văn có dạng bảng rộng (mỗi dòng một trạm và một thời điểm, mỗi cột một cảm biến); số đo đến vài phút một lần, số đo trễ hoặc được hiệu chỉnh đến sau vài giờ và phải ghi đè giá trị cũ, dữ liệu của trạm hỏng bị xóa cả ngày. Đây là workload *update-heavy*, khác TPC-DS. Đồng thời các cơ quan và trường đại học ở Việt Nam thường chỉ có 1–4 máy chủ, trong khi các so sánh hiện có chạy TPC-DS 1–3 TB trên cụm đám mây [3, 4].
- *Hướng tiếp cận:* xây bộ sinh workload có kiểm soát từ dữ liệu thủy văn thật, cho mọi format ghi cùng một luồng thao tác giống hệt byte, đo chi phí ghi, đọc, compaction, metadata và time travel, và kiểm tra tính đúng đắn của mọi lần chạy.
- *RQ1:* với tỷ lệ cập nhật trễ $u \in \{0, 1, 5, 20\}\%$, format và chế độ ghi nào có tổng chi phí (nạp + compaction + truy vấn) thấp nhất?
- *RQ2:* chu kỳ compaction nào cân bằng tốt nhất giữa số file nhỏ và độ trễ đọc?
- *RQ3:* metadata tăng thế nào theo số commit và ảnh hưởng ra sao đến time travel?
- *Giả thuyết:* (H1) ở u thấp, khác biệt chủ yếu nằm ở đường đọc như LHBench; (H2) ở u cao, MoR giảm chi phí ghi nhưng tăng độ trễ đọc cho đến khi compaction; (H3) tồn tại "điểm gãy" về số file nhỏ mà sau đó compaction định kỳ có lợi rõ rệt.
- *Contributions:* (C1) bộ sinh workload sensor-LST mã nguồn mở, tất định theo seed, có ground truth; (C2) so sánh có kiểm soát, tái lập được của ba format (cộng Parquet thuần) ở hai chế độ ghi và ba chu kỳ compaction, 228 lần chạy có lặp và xen kẽ ngẫu nhiên; (C3) giải thích kết quả bằng thiết kế của từng format, danh sách các "bẫy" khi dùng thực tế và bảng khuyến nghị; (C4) demo trực tiếp trên localhost.

2. Related Work

- *Lakehouse và table format:* cả ba format lưu dữ liệu thành file Parquet bất biến và giữ riêng một mô tả có phiên bản về các file tạo nên bảng — tương tự các run của LSM-tree [9], nên còn được gọi là log-structured tables [4]. Delta ghi mỗi commit thành file JSON trong `_delta_log` và checkpoint Parquet mỗi 10 commit [2]; Iceberg ghi cây metadata JSON →

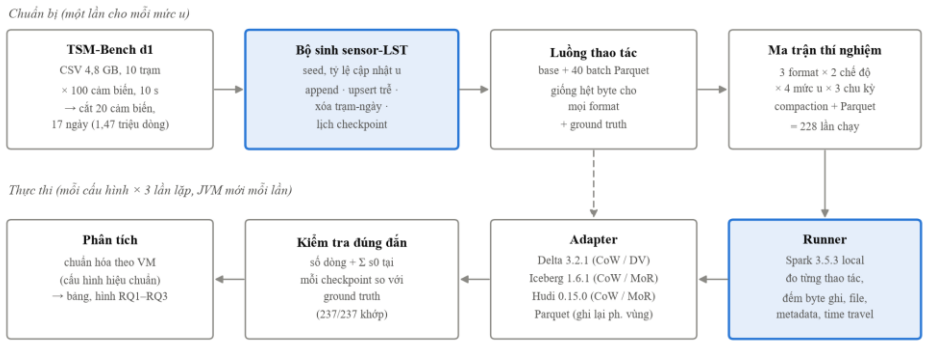
manifest list → manifest Avro cho mỗi snapshot [15]; Hudi tổ chức file group gồm base file và log file, có timeline các instant, chỉ mục khóa và cơ chế small-file handling [16].

- *Ghi thay đổi cấp dòng:* CoW ghi lại mọi file chứa dòng bị đổi; MoR ghi thay đổi rẻ rồi hợp nhất khi đọc — Iceberg dùng position-delete file [15], Hudi ghi log file [16], Delta không có MoR kiểu log mà dùng deletion vectors đánh dấu dòng đã xóa trong bitmap [17].
- *Benchmark Lakehouse:* LHBench [3] chạy TPC-DS 3 TB và kết luận khác biệt chủ yếu do thời gian đọc dữ liệu, Hudi nạp chậm hơn vì tối ưu cho upsert theo khóa; LST-Bench [4] mở rộng TPC-DS với các mẫu bảo trì, tối ưu, time travel và chỉ số ổn định hiệu năng; AutoComp [5] cho thấy small-file là vấn đề vận hành lớn ở LinkedIn; Harby và Zulkernine [6] khảo sát data lake, warehouse và lakehouse; TPC-DS [8] vẫn là workload chuẩn.
- *Dữ liệu chuỗi thời gian:* Jensen và cộng sự [10] khảo sát các hệ quản trị chuỗi thời gian; TSM-Bench [7] đánh giá 7 TSDB với truy vấn từ bài toán giám sát nước. Đề tài dùng lại dữ liệu và phong cách truy vấn của TSM-Bench nhưng thêm hiệu chỉnh trễ và xóa — điều mà mạng quan trắc thật gặp phải.
- *Phương pháp benchmark trên cloud:* hiệu năng trên máy dùng chung biến động theo thời gian và giữa các máy cùng loại [11, 12]; cách khắc phục là lặp nhiều lần, xen kẽ ngẫu nhiên các hệ được so sánh trên cùng máy, báo cáo độ phân tán và dùng cấu hình hiệu chuẩn.
- *Vị trí của đề tài:* chưa có nghiên cứu công bố nào kết hợp workload cảm biến cập nhật nhiều, cả hai chế độ ghi, chu kỳ compaction có kiểm soát và phần cứng một node.

---

3. Proposed Method

Hình 1 tóm tắt kiến trúc: phần chuẩn bị sinh luồng thao tác một lần cho mỗi mức *u*; phần thực thi phát lại luồng đó trên từng layout trong một JVM mới và kiểm tra với ground truth tại mỗi checkpoint.



Hình 1. Kiến trúc hệ thống thí nghiệm. Đường nét đứt: luồng thao tác đã sinh được mọi adapter đọc như nhau.

1. **Chuẩn bị dữ liệu:** đọc thẳng file nén của TSM-Bench d1 (CSV 4,8 GB) theo dòng, không giải nén ra đĩa; giữ 10 trạm × 20 cảm biến × 17 ngày (1,47 triệu dòng, 253 MB Parquet); thêm cột `day` (phân vùng) và `ingest_batch`.
2. **Bộ sinh workload sensor-LST:** nạp 7 ngày lịch sử, rồi 40 micro-batch, mỗi batch 6 giờ dữ liệu mới (21.600 dòng) cộng u × 21.600 bản ghi hiệu chỉnh trễ — các khóa đã tồn tại, chọn ngẫu nhiên trong 72 giờ trước, nhận giá trị mới; cứ 10 batch xóa một trạm-ngày. Toàn bộ luồng được ghi sẵn thành Parquet trước khi chạy, nên mọi format nhận đầu vào giống hệt byte; bộ sinh đồng thời tính trạng thái mong đợi (số dòng và tổng s0) tại mỗi checkpoint.
3. **Adapter cho bốn layout:** cùng một giao diện create / append / upsert / delete / compact / time travel / thống kê (Bảng 1). MERGE và DELETE luôn kèm vị từ `day IN (…)` để cắt tỉa phân vùng; với Hudi, khóa bản ghi là chuỗi `id_station|epoch_micros` (xem mục 6.4).

Bảng 1. Cơ chế ghi và bảo trì của từng format

	Delta Lake 3.2.1	Iceberg 1.6.1	Hudi 0.15.0
Nhật ký phiên bản	JSON commit + checkpoint Parquet mỗi 10 commit	metadata JSON → manifest list → manifest	timeline instant + metadata table
Thay đổi cấp dòng (MoR)	deletion vector (bitmap)	position-delete file	log file trong file group
Khóa bản ghi	không (MERGE theo vị từ)	không (MERGE theo vị từ)	bắt buộc; chỉ mục khóa
Compaction	REORG … APPLY (PURGE) + OPTIMIZE	rewrite_data_files (+ rewrite_position_delete_files)	run_compaction (MoR) / run_clustering (CoW)
Giới hạn time travel (mặc định)	đến khi VACUUM	đến khi expire snapshot	chỉ timeline đang hoạt động

4. **Runner và đo lường:** mỗi lần chạy là một tiến trình với JVM Spark mới; bấm giờ quanh hành động Spark hoàn tất mỗi thao tác (gồm lập kế hoạch, thực thi, commit); sau mỗi thao tác liệt kê thư mục bảng để tính byte dữ liệu và metadata đã ghi (bỏ file `.crc` vì không có trên object storage). Write amplification và tốc độ suy giảm theo (1)–(2):

$$\mathrm{WA} = \frac{B_{\mathrm{written}}}{B_{\mathrm{logical}}} \tag{1}$$

$$D = \frac{L_{\mathrm{last}} - L_{\mathrm{first}}}{L_{\mathrm{first}}} \tag{2}$$

Trong đó B_{written} là số byte file dữ liệu đã ghi, B_{logical} là số byte của bảng cuối ghi một lần bằng Parquet snappy; D là độ suy giảm, L là độ trễ warm của Q1–Q5 tại checkpoint.

5. **Kiểm tra đúng đắn:** tại mỗi checkpoint so số dòng và Σ s0 của bảng với ground truth của bộ sinh; một lần chạy chỉ được tính khi khớp ở cả bốn checkpoint.
6. **Kiểm soát công bằng:** một engine (Spark 3.5.3), nén snappy cho mọi format (mặc định Iceberg là zstd, Hudi là gzip), không dọn file cũ trong lúc chạy (byte trên đĩa = byte đã ghi), xáo ngẫu nhiên thứ tự các cấu hình trong mỗi vòng lặp để các format xen kẽ trên cùng máy.
7. **Demo:** ứng dụng Streamlit trên localhost (`run_demo.bat`): mỗi format chạy trong một tiến trình Spark riêng; người dùng bấm ghi batch, xóa trạm, compaction, xem số file, metadata, thời gian, và time travel về phiên bản bất kỳ, kể cả đọc lại trạm đã bị xóa.

Điểm khác so với LHBench / LST-Bench: workload cảm biến có hiệu chỉnh trễ và xóa thay cho TPC-DS; phần cứng một node; điều khiển chu kỳ compaction như một biến; kiểm tra ground truth ở mọi checkpoint; chuẩn hóa thời gian giữa các máy bằng cấu hình hiệu chuẩn.

---

4. Dataset

- *Nguồn dữ liệu:* TSM-Bench d1 [7] gồm 10 trạm × 100 cảm biến, mẫu 10 giây, 60 ngày (5,18 triệu dòng), sinh từ chuỗi nhiệt độ và mực nước của Cơ quan Môi trường Liên bang Thụy Sĩ. Giá trị trong d1 đã chuẩn hóa, gần như phân phối đều trên [0, 1) và ít tự tương quan nên nén kém — làm tăng byte tuyệt đối nhưng ảnh hưởng như nhau tới mọi format. Bảng 2 tóm tắt các tham số workload.

Bảng 2. Tham số workload (quy mô small dùng trong mọi thí nghiệm)

Tham số	Giá trị
Trạm × cảm biến	10 × 20 (TSM-Bench d1), mẫu 10 giây
Lịch sử nạp ban đầu	7 ngày — 604.800 dòng
Micro-batch	40 batch × 6 giờ = 10 ngày; 21.600 dòng mới mỗi batch
Tỷ lệ cập nhật trễ *u*	0, 1, 5, 20% (0–4.320 bản ghi mỗi batch), khóa trong 72 giờ trước
Xóa / checkpoint	xóa một trạm-ngày mỗi 10 batch; checkpoint mỗi 10 batch
Chu kỳ compaction	không bao giờ, mỗi 5 batch, mỗi 20 batch
Bảng cuối	1,43 triệu dòng, khoảng 222 MB Parquet

- *Truy vấn:* năm truy vấn theo phong cách giám sát của TSM-Bench (Bảng 3), mỗi truy vấn chạy hai lần tại mỗi checkpoint (cold, warm); time travel đọc phiên bản 5 commit trước và phiên bản đầu tiên.

Bảng 3. Các truy vấn phân tích

Truy vấn	Nội dung	Phạm vi dữ liệu
Q1	Trung bình theo giờ của mỗi trạm	24 giờ gần nhất
Q2	Top-5 trạm theo giá trị đỉnh	72 giờ gần nhất
Q3	Trung bình theo ngày và vùng (join bảng metadata trạm)	toàn bộ lịch sử
Q4	Một cảm biến của một trạm trong 2 giờ	1–2 phân vùng ngày
Q5	count(*) và Σ s0 (đồng thời là checksum)	toàn bộ lịch sử

---

5. Experimental Setup

- *Ma trận:* 3 format × 2 chế độ × 4 mức *u* × 3 chu kỳ compaction = 72 cấu hình, cộng 4 baseline Parquet thuần (ghi lại phân vùng ngày bị ảnh hưởng khi upsert/xóa); mỗi cấu hình lặp 3 lần — 228 lần chạy, chạy theo vòng, thứ tự trong vòng xáo ngẫu nhiên có seed.
- *Phiên bản:* Spark 3.5.3 [13, 14] (local, toàn bộ lõi, driver 60% RAM), Delta Lake 3.2.1, Iceberg 1.6.1 (Hadoop catalog, format v2), Hudi 0.15.0; Java 21, Python 3.13 (cả bốn layout đã được kiểm tra cho kết quả giống hệt trên Java 11/21 và Python 3.12/3.13).
- *Phần cứng:* notebook CPU của Kaggle (4 vCPU, 31,3 GB RAM, Linux, đĩa cục bộ), chạy nền; mỗi mức *u* là một notebook riêng trên một VM riêng, mất 2–6 giờ.
- *Hiệu chuẩn máy:* mỗi VM chạy lặp cấu hình Delta CoW, $u = 0$. Cùng cấu hình mất 65, 103, 70 và 98 s trên VM của các mức $u = 0, 1, 5, 20\%$ — hai VM chậm hơn khoảng 1,5 lần. So sánh giữa các format luôn trong cùng một VM; so sánh giữa các mức *u* dùng thời gian chia cho hệ số của VM (1,00; 1,58; 1,07; 1,50).
- *Metrics:* thời gian từng thao tác, thời gian compaction, độ trễ truy vấn cold/warm, write amplification, số file dữ liệu và file delete/log còn hiệu lực, byte và số file metadata, số phiên bản, độ trễ time travel, tốc độ suy giảm; báo cáo median và hệ số biến thiên (CV) qua 3 lần lặp.

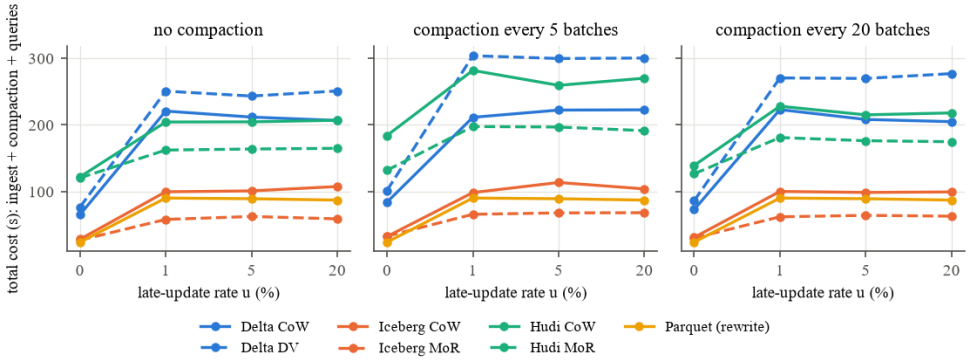
---

6. Results & Ablation

Ngoài ma trận chính, mục này có ba so sánh theo kiểu ablation, mỗi lần tắt một cơ chế để đo phần đóng góp của nó: tắt compaction (mục 6.2), tắt deletion vectors của Delta bằng cách dùng CoW (mục 6.1 và 6.2), và bỏ chuẩn hóa theo VM (mục 6.5).

Bảng 4. Kết quả ở u = 20%, không compaction và Iceberg MoR với compaction mỗi 20 batch (median 3 lần, giây quy về máy tham chiếu). Đọc = Q1–Q5 warm ở checkpoint cuối; File = dữ liệu + delete/log còn hiệu lực

Layout	Chu kỳ	Nạp	Compaction (s)	Truy vấn	Tổng	WA	Đọc	File
Iceberg MoR	–	51	0	8	**59**	1,16	2,24	167+154
Parquet (ghi lại)	–	84	0	3	87	9,53	0,70	84+0
Iceberg CoW	–	105	0	3	108	9,37	0,63	17+0
Hudi MoR	–	154	0	11	165	2,51	2,87	5+124
Delta CoW	–	200	0	7	207	9,37	1,64	17+0
Hudi CoW	–	204	0	3	207	11,00	0,80	11+0
Delta DV	–	233	0	19	251	1,16	4,52	167+154
Iceberg MoR	mỗi 20	51	8	4	**63**	2,34	0,65	17+10



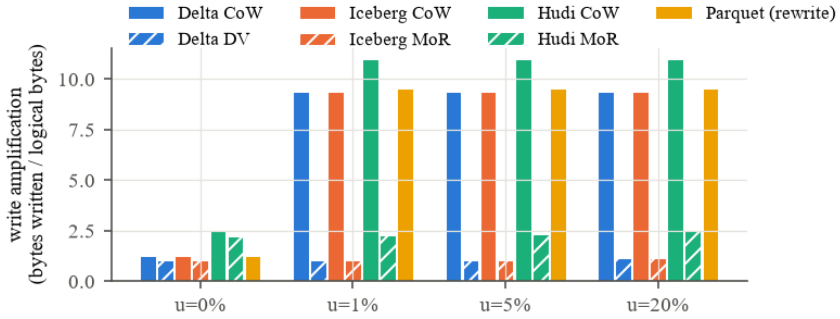
Hình 2. Tổng chi phí (nạp + compaction + mọi truy vấn checkpoint; median 3 lần, chuẩn hóa theo VM) theo tỷ lệ cập nhật trễ. Nét liền: CoW; nét đứt: MoR / deletion vectors.

6.1 RQ1 – Tỷ lệ cập nhật và tổng chi phí

Bước nhảy nằm ở chỗ có hay không có dữ liệu trễ, không nằm ở giá trị u. Chuyển từ chỉ append sang có dữ liệu trễ làm tổng chi phí tăng 3,4–3,8 lần với các layout CoW (Delta 3,4; Iceberg 3,5; Parquet 3,8), 3,3 lần với Delta DV, 2,1 lần với Iceberg MoR và 1,3–1,7 lần với Hudi (Hình 2). Từ u = 1% đến u = 20%, chi phí không compaction của mọi layout chênh nhau tối đa 8%. Ở u = 1%, mỗi batch đã có 216 bản ghi hiệu chỉnh rải trên 72 giờ, tức trên bốn phân vùng ngày chỉ có vài file mỗi phân vùng, nên hầu như mọi file của các phân vùng đó đều bị chạm tới. CoW vì vậy ghi lại đúng những file ấy dù 216 hay 4.320 dòng thay đổi.

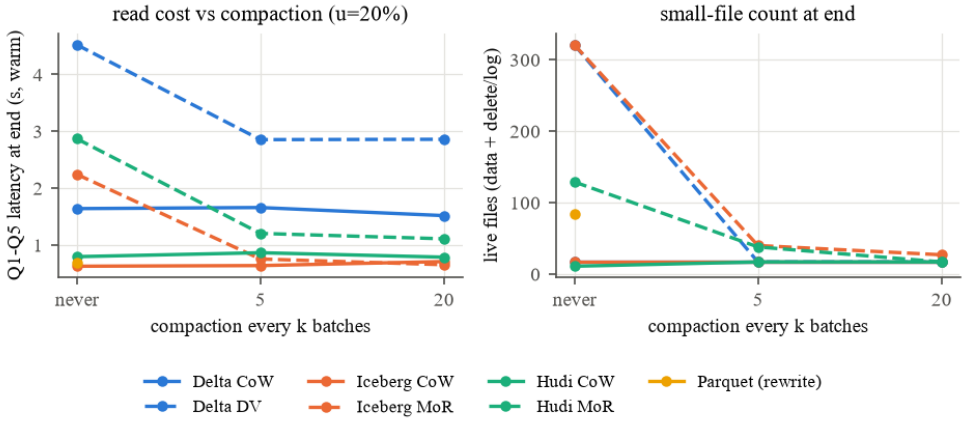
Iceberg MoR rẻ nhất ở mọi $u > 0$ (58–63 s; chi tiết ở u = 20% trong Bảng 4), so với 1,4–1,6 lần ở Parquet thuần, 1,6–1,8 lần ở Iceberg CoW, 2,6–2,8 lần ở Hudi MoR và 3,3–4,3 lần ở các cấu hình Delta và Hudi CoW. Ở u = 0, Parquet thuần nhanh nhất (24 s), tiếp theo là Iceberg (27–28 s);

Hudi chậm nhất (120–122 s) do small-file handling gộp bản ghi mới vào base file có sẵn. Write amplification (Hình 3) cho thấy cơ chế: mọi layout CoW ghi khoảng 9,4 lần kích thước bảng (Hudi CoW 11,0) khi $u > 0$, trong khi Delta DV và Iceberg MoR chỉ 1,03–1,16. Nhưng byte không phải là thời gian: Delta DV ghi ít byte hơn Delta CoW 8 lần mà nạp chậm hơn 9–16%, còn Iceberg MoR biến khoản tiết kiệm đó thành tốc độ nạp nhanh gấp đôi Iceberg CoW. Giả thuyết H1 không đúng với workload này: ở quy mô gigabyte, các format khác nhau chủ yếu ở đường *ghi*.



Hình 3. Write amplification khi không compaction.

6.2 RQ2 – Chu kỳ compaction

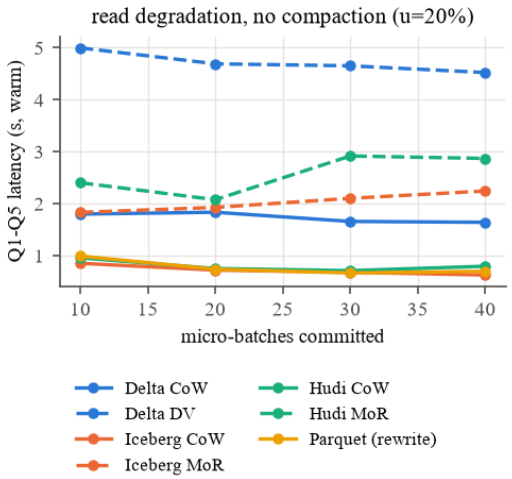


Hình 4. Độ trễ đọc cuối lần chạy và số file còn hiệu lực theo chu kỳ compaction (u = 20%).

Không compaction, MoR trả giá khi đọc (Hình 4). Sau 40 batch ở u = 20%, Delta DV và Iceberg MoR giữ 167 file dữ liệu cộng 154 deletion vector hoặc delete file; Hudi MoR giữ 5 base file và 124 log file; CoW chỉ 11–17 file. Độ trễ warm Q1–Q5 tương ứng 2,24 s (Iceberg MoR), 2,87 s (Hudi MoR), 4,52 s (Delta DV), so với 0,63; 0,80 và 1,64 s ở chế độ CoW của cùng format. Theo thời gian (Hình 5), độ trễ CoW giảm nhẹ khi JVM nóng lên (−9% đến −26%), còn Iceberg MoR tăng 24% và Hudi MoR tăng 15%; Hudi MoR nhảy lên giữa batch 20 và 30 — đó là "điểm gãy" về số file nhỏ (H3) trong dữ liệu.

Compaction mỗi 20 batch là đủ: Iceberg MoR còn 0,65 s và Hudi MoR 1,11 s, trong vòng 4–40% so với CoW, với 8 s (Iceberg) và 17 s (Hudi) compaction cho cả lần chạy. Compaction mỗi 5 batch không đọc nhanh hơn (0,75 và 1,20 s) mà tốn gấp đôi (16 và 36 s). Với CoW, compaction

không đem lại gì khi $u > 0$ vì mỗi batch đã ghi lại phân vùng bị chạm thành một file. H2 đúng với Iceberg và Hudi nhưng không đúng với Delta.



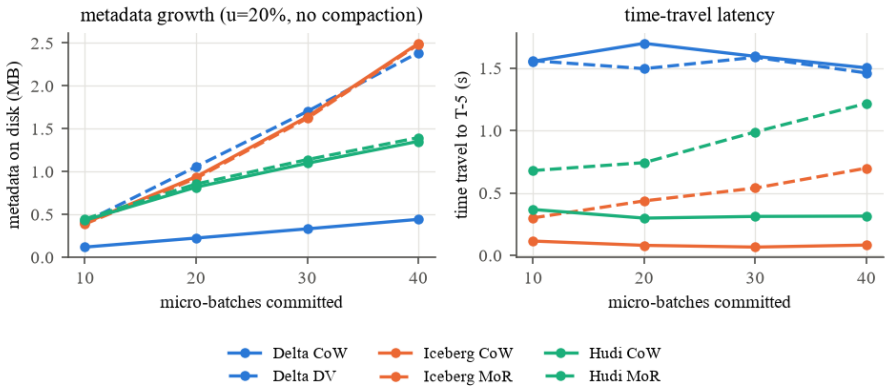
Hình 5. Độ trễ warm Q1–Q5 trong suốt lần chạy, không compaction (u = 20%).

Bảng 5. Độ trễ warm từng truy vấn ở checkpoint cuối, u = 20% (giây, median 3 lần)

Layout (compaction)	Q1	Q2	Q3	Q4	Q5
Iceberg CoW (không)	0,09	0,10	0,28	0,05	0,10
Iceberg MoR (không)	0,14	0,22	1,07	0,07	0,75
Iceberg MoR (mỗi 20)	0,12	0,11	0,26	0,06	0,08
Hudi CoW (không)	0,15	0,12	0,37	0,06	0,10
Hudi MoR (không)	0,15	0,28	1,28	0,16	0,95
Hudi MoR (mỗi 20)	0,14	0,20	0,47	0,08	0,22
Delta CoW (không)	0,28	0,29	0,54	0,25	0,33
Delta DV (không)	0,87	0,84	1,46	0,66	0,82
Delta DV (mỗi 20)	0,54	0,46	0,85	0,46	0,51
Parquet (ghi lại)	0,12	0,13	0,27	0,06	0,10

Bảng 5 cho thấy truy vấn nào chịu thiệt của MoR. Với Iceberg và Hudi, thiệt hại dồn vào các truy vấn quét toàn bộ lịch sử: Q3 chậm hơn CoW 3,5–3,9 lần và Q5 chậm hơn 7,8–9,7 lần vì phải mở mọi file dữ liệu kèm delete/log file; các truy vấn kiểu dashboard trên dữ liệu gần đây (Q1, Q4) gần như không bị ảnh hưởng nhờ cắt tỉa phân vùng. Delta DV thì khác: cả năm truy vấn đều chậm hơn Delta CoW 2,5–3,1 lần, kể cả sau khi đã xóa hết deletion vector (2,86 so với 1,52 s với cùng 17 file) — chỉ riêng việc bật tính năng này đã làm chậm mỗi truy vấn khoảng 0,2–0,3 s.

6.3 RQ3 – Metadata và time travel

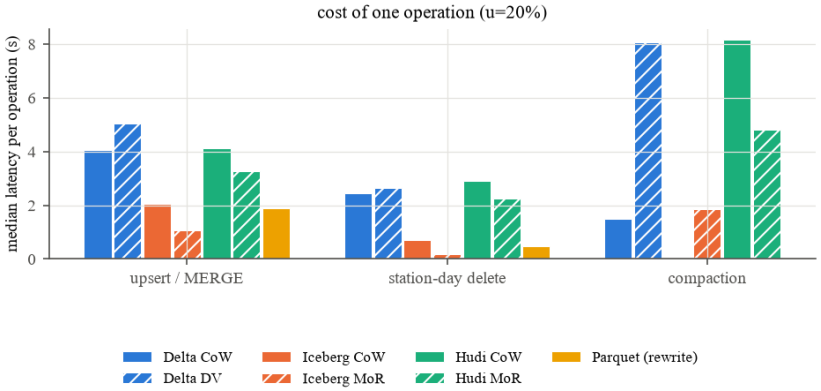


Hình 6. Metadata trên đĩa và độ trễ time travel về phiên bản 5 commit trước, theo batch (*u* = 20%, không compaction).

Metadata tăng tuyến tính nhưng với tốc độ rất khác nhau (Hình 6). Mỗi micro-batch, log của Delta CoW tăng 11 KB (0,45 MB sau 45 phiên bản), Hudi 31–32 KB (1,4 MB), Delta DV 67 KB và Iceberg 72 KB ở cả hai chế độ (2,4–2,5 MB). Iceberg ghi metadata JSON, manifest list và manifest mới cho mỗi snapshot; log của Delta DV tăng nhanh gấp sáu lần Delta CoW vì mỗi commit thêm lại mọi file bị chạm kèm deletion vector mới, và bảng DV giữ nhiều file gấp mười lần (167 so với 17). Metadata của Hudi vẫn tăng sau khi timeline đầy, nhưng chỉ 23 phiên bản còn time travel được: các instant cũ bị archive kể cả khi đã tắt cleaner.

Độ trễ time travel đi theo số delete/log file chứ không theo kích thước metadata. Đọc phiên bản 5 commit trước mất 0,06–0,12 s với Iceberg CoW và khoảng 0,3 s với Hudi CoW, không đổi suốt lần chạy; khoảng 1,5 s với Delta ở cả hai chế độ, cũng không đổi nhờ checkpoint log mỗi 10 commit. Hai format MoR tích lũy delete thì chậm dần: Iceberg MoR từ 0,30 lên 0,70 s, Hudi MoR từ 0,68 lên 1,22 s.

6.4 Phân tích theo thao tác, theo thời gian và các "bẫy" khi dùng thực tế



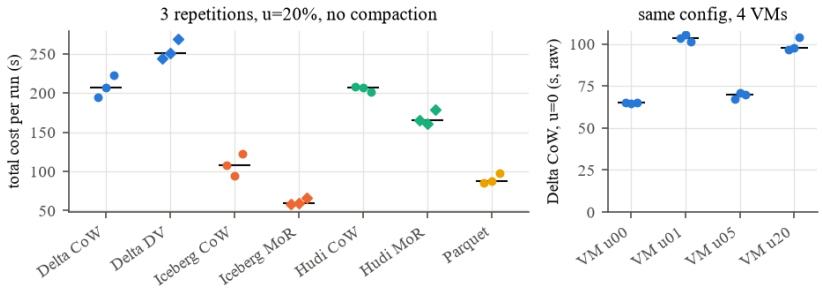
Hình 7. Độ trễ median của một thao tác ở *u* = 20%: một upsert micro-batch và một lần xóa trạm-ngày (không compaction), một lần compaction (cả hai chu kỳ).

- **Một thao tác:** theo Hình 7, MERGE một batch có 4.320 bản ghi hiệu chỉnh mất 1,06 s với Iceberg MoR, 2,04 s với Iceberg CoW, 1,88 s với Parquet, 3,3–4,1 s với Hudi và 4,0–5,1 s

với Delta. Xóa một trạm-ngày rẻ nhất ở Iceberg MoR (0,19 s, một delete file), 2,2–2,9 s ở Delta và Hudi. Nạp ban đầu 604.800 dòng mất 5,6–6,3 s (Parquet, Iceberg), 10,5 s (Delta), 12,9–13,2 s (Hudi) — nhất quán với nhận xét Hudi nạp chậm của LHBench [3].

- **Theo thời gian:** độ trễ nạp không tăng khi bảng lớn dần từ 0,6 lên 1,43 triệu dòng vì vị từ `day IN (…)` giới hạn mỗi MERGE trong bốn phân vùng mới nhất. Append của Delta chậm hẳn ở mỗi commit thứ 10 (2,0–5,1 s so với khoảng 0,6 s) vì commit đó ghi checkpoint log đồng bộ; append của Hudi luôn khoảng 2,2–2,4 s, gấp 7 lần Iceberg.
- **Bẫy của Hudi:** khóa ghép chứa timestamp bị `bulk_insert` và `upsert` biểu diễn khác nhau nên upsert chèn bản ghi trùng (chỉ checksum phát hiện; đề tài dùng khóa chuỗi `id_station|epoch_micros`); compaction mặc định chỉ được lập lịch sau 5 delta commit; archive chỉ giữ khoảng 23 instant cho time travel.
- **Bẫy của Iceberg và mặc định chung:** `rewrite_data_files` của Iceberg 1.6 để lại delete file hết tác dụng (23 file với compaction mỗi 5 batch, Delta để lại 0); Iceberg mặc định nén zstd, Hudi gzip, Delta snappy — giữ mặc định là đo cả bộ nén.

6.5 Độ biến thiên và hiệu chuẩn



Hình 8. Biến thiên giữa các lần lặp và giữa các máy. Trái: tổng chi phí từng lần lặp (vạch = median). Phải: cấu hình hiệu chuẩn trên bốn VM, giây thô.

Cả 237 lần chạy (228 cộng 9 lần hiệu chuẩn) đều khớp số dòng và checksum ở cả bốn checkpoint. Hệ số biến thiên median của tổng chi phí qua 3 lần lặp là 3,5% (tối đa 12,7%, Iceberg CoW ở $u = 20\%$), nên chênh lệch trên khoảng 15% nằm ngoài nhiễu. Ngược lại (Hình 8, bên phải), cùng một cấu hình mất 65 đến 103 s trên các VM cùng loại — chênh 1,6 lần — sẽ làm sai mọi so sánh giữa các mức u nếu không chuẩn hóa.

---

7. Rủi ro & hạn chế

- **Một node, đĩa cục bộ:** chưa có mạng và object storage, nơi mỗi file là một request — chi phí file nhỏ ở đây là cận dưới. Giảm thiểu: chạy lại trên cụm 3–4 node với MinIO (hướng phát triển).

- **Máy dùng chung trên cloud:** tốc độ VM chênh tới 1,58 lần. Giảm thiểu: lặp 3 lần, xen kẽ ngẫu nhiên, cấu hình hiệu chuẩn, chỉ so sánh format trong cùng VM; chuẩn hóa giả định mọi layout chậm đi như nhau trên VM chậm.
- **Quy mô dữ liệu nhỏ:** bảng khoảng 222 MB, không chạy TPC-DS SF10–SF100 như đề cương ban đầu vì giới hạn phần cứng. Giảm thiểu: báo cáo rõ phạm vi; workload sensor-LST là đóng góp chính.
- **Dữ liệu d1 gần như ngẫu nhiên đều:** nén kém hơn dữ liệu cảm biến thật. Giảm thiểu: chỉ so sánh tỷ lệ giữa các format, không so sánh byte tuyệt đối với hệ thống thật.
- **Phụ thuộc phiên bản và cấu hình:** kết quả đúng cho các phiên bản đã chốt; chỉ thay đổi bộ nén, việc dọn file và lịch compaction của Hudi, mọi thiết lập được công bố trong repo.
- **Một engine:** chỉ Spark; Trino hoặc DuckDB có thể ưu tiên format khác. Giảm thiểu: đọc đa engine là hướng phát triển.
- **Số lần lặp:** đề cương đề nghị ít nhất 5 lần chạy; đề tài lặp 3 lần vì mỗi mức u đã mất 2–6 giờ. CV median 3,5% đủ để phân biệt chênh lệch trên khoảng 15%, nhưng p95 qua 3 lần chỉ mang tính tham khảo: ở u = 20%, không compaction, p95 tổng chi phí là 65 s (Iceberg MoR), 178 s (Hudi MoR) và 267 s (Delta DV).
- **Nhịp micro-batch:** đề cương đề nghị 1–5 phút; mỗi batch ở đây chứa 6 giờ dữ liệu (21.600 dòng) để 40 batch phủ đủ 10 ngày. Kết quả phản ánh chi phí theo kích thước batch, chưa mô phỏng tần suất ghi vài phút một lần.
- **Biến và baseline chưa thử:** chỉ phân vùng theo ngày, chưa thử phân vùng trạm + ngày, kích thước file đích và Z-order; chưa chạy baseline cấu hình mặc định vì đã thống nhất nén snappy. Mỗi phân vùng ngày chỉ khoảng 13 MB nên mọi file đều dưới 128 MB, số file nhỏ chính là cột File trong Bảng 4.
- **LST-Bench:** bộ sinh sensor-LST chạy độc lập, chưa được tích hợp thành workload của LST-Bench.

---

8. Kết luận & hướng phát triển

Với bảng cảm biến cập nhật nhiều trên một node, việc có dữ liệu trễ — chứ không phải tỷ lệ — quyết định chi phí: tổng chi phí tăng 1,3–3,8 lần khi xuất hiện hiệu chỉnh trễ rồi đứng yên trong 8% từ 1% đến 20% khi không compaction (RQ1). Iceberg MoR là layout rẻ nhất ở mọi $u > 0$, và compaction mỗi 20 micro-batch đưa Iceberg và Hudi MoR về gần độ trễ đọc của CoW với 8–17 s, còn compaction mỗi 5 batch chỉ tốn thêm (RQ2). Metadata tăng tuyến tính 11–72 KB mỗi batch mà không ảnh hưởng độ trễ time travel — vốn đi theo số delete/log file; archive mặc định của Hudi giới hạn time travel ở khoảng 23 phiên bản (RQ3). Bảng 6 tóm tắt khuyến nghị cho một hệ thống giám sát quy mô nhỏ.

Bảng 6. Khuyến nghị cấu hình cho hệ thống giám sát nhỏ (một node, vài GB mỗi ngày)

Tình huống	Layout	Compaction
Chỉ append ($u = 0$), cần ACID / lịch sử	Iceberg (chế độ nào cũng được)	mỗi 20 batch
Chỉ append, không cần ACID	Parquet thuần	–
Có dữ liệu trễ (u = 1–20%)	**Iceberg MoR**	mỗi 20 batch
Đã dùng Delta Lake	Delta CoW (tắt deletion vectors)	tùy chọn
Đã dùng Hudi	Hudi MoR	mỗi 20 batch
Cần time travel dài	Iceberg hoặc Delta (Hudi giữ ≈23 phiên bản)	–

1. Chạy trên cụm 3–4 node với object storage MinIO để đo ảnh hưởng của request và listing đối với file nhỏ và metadata.
2. Đọc đa engine: Trino, DuckDB đọc Iceberg; thêm catalog (Apache Polaris, Unity Catalog).
3. Chính sách compaction theo ngân sách kiểu AutoComp [5], kích hoạt theo số delete/log file thay vì theo chu kỳ cố định.
4. Bổ sung TPC-DS SF10 theo giao thức LHBench / LST-Bench để có điểm neo so sánh trực tiếp.

Caveats

- Thời gian đo trên VM Kaggle dùng chung; số liệu giữa các mức u đã chuẩn hóa bằng hệ số hiệu chuẩn, chỉ dùng để so sánh tương đối. Mọi kết luận gắn với Spark 3.5.3, Delta 3.2.1, Iceberg 1.6.1, Hudi 0.15.0 và cấu hình công bố trong repo.
- Dữ liệu là bản thu nhỏ của TSM-Bench d1 (10 trạm × 20 cảm biến × 17 ngày), không phải dữ liệu trạm quan trắc thật của Việt Nam.
- Demo localhost dùng quy mô nhỏ hơn (4 trạm, 12 batch); con số thời gian trong demo chỉ minh họa.

References

[1] Armbrust, M., Ghodsi, A., Xin, R., Zaharia, M.: Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. CIDR 2021 (2021)

[2] Armbrust, M., et al.: Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. PVLDB 13(12), 3411–3424 (2020)

[3] Jain, P., Kraft, P., Power, C., Das, T., Stoica, I., Zaharia, M.: Analyzing and Comparing Lakehouse Storage Systems. CIDR 2023 (2023) — https://www.cidrdb.org/cidr2023/papers/p92-jain.pdf

[4] Camacho-Rodríguez, J., et al.: LST-Bench: Benchmarking Log-Structured Tables in the Cloud. Proc. ACM Manag. Data 2(1) (2024). doi:10.1145/3639314

[5] Gruenheid, A., et al.: AutoComp: Automated Data Compaction for Log-Structured Tables in Data Lakes. SIGMOD-Companion 2025 (2025). doi:10.1145/3722212.3724430

[6] Harby, A.A., Zulkernine, F.: Data Lakehouse: A Survey and Experimental Study. Information Systems 127 (2025)

[7] Khelifati, A., Khayati, M., Dignös, A., Difallah, D., Cudré-Mauroux, P.: TSM-Bench: Benchmarking Time Series Database Systems for Monitoring Applications. PVLDB 16(11), 3363–3376 (2023). doi:10.14778/3611479.3611532

[8] Nambiar, R.O., Poess, M.: The Making of TPC-DS. VLDB 2006 (2006)

[9] O'Neil, P., Cheng, E., Gawlick, D., O'Neil, E.: The Log-Structured Merge-Tree (LSM-Tree). Acta Informatica 33(4), 351–385 (1996)

[10] Jensen, S.K., Pedersen, T.B., Thomsen, C.: Time Series Management Systems: A Survey. IEEE Trans. Knowl. Data Eng. 29(11), 2581–2600 (2017)

[11] Leitner, P., Cito, J.: Patterns in the Chaos — A Study of Performance Variation and Predictability in Public IaaS Clouds. ACM Trans. Internet Technol. 16(3) (2016)

[12] Uta, A., et al.: Is Big Data Performance Reproducible in Modern Cloud Networks? NSDI 2020 (2020)

[13] Zaharia, M., et al.: Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. NSDI 2012 (2012)

[14] Armbrust, M., et al.: Spark SQL: Relational Data Processing in Spark. SIGMOD 2015 (2015)

[15] Apache Iceberg: Iceberg Table Spec — https://iceberg.apache.org/spec/ (truy cập 10/2026)

[16] Apache Hudi: Table & Query Types — https://hudi.apache.org/docs/table_types (truy cập 10/2026)

[17] Delta Lake: Deletion Vectors — https://docs.delta.io/latest/delta-deletion-vectors.html (truy cập 10/2026)